

VersaSlot: Efficient Fine-grained FPGA Sharing with Big.Little Slots and Live Migration in FPGA Cluster

Jianfeng Gu, Hao Wang, Xiaorang Guo, Martin Schulz and Michael Gerndt

Chair of Computer Architecture and Parallel Systems, Technical University of Munich

Email: {jianfeng.gu, whhao.wang, xiaorang.guo}@tum.de, {schulzm, gerndt}@in.tum.de

Abstract—As FPGAs gain popularity for on-demand application acceleration in data center computing, dynamic partial reconfiguration (DPR) has become an effective fine-grained sharing technique for FPGA multiplexing. However, current FPGA sharing encounters partial reconfiguration contention and task execution blocking problems introduced by the DPR, which significantly degrade application performance. In this paper, we propose VersaSlot, an efficient spatio-temporal FPGA sharing system with novel Big.Little slot architecture that can effectively resolve the contention and task blocking while improving resource utilization. For the heterogeneous Big.Little architecture, we introduce an efficient slot allocation and scheduling algorithm, along with a seamless cross-board switching and live migration mechanism, to maximize FPGA multiplexing across the cluster. We evaluate the VersaSlot system on an FPGA cluster composed of the latest Xilinx UltraScale+ FPGAs (ZCU216) and compare its performance against four existing scheduling algorithms. The results demonstrate that VersaSlot achieves up to 13.66x lower average response time than the traditional temporal FPGA multiplexing, and up to 2.19x average response time improvement over the state-of-the-art spatio-temporal sharing systems. Furthermore, VersaSlot enhances the LUT and FF resource utilization by 35% and 29% on average, respectively.

I. INTRODUCTION

Field Programmable Gate Arrays (FPGAs) have rapidly gained popularity in edge and data center computing, offering efficient application acceleration with higher flexibility and lower power consumption [1]–[5]. Leading cloud providers [6]–[8] have integrated FPGAs into their platforms, allowing users to develop highly customized and on-demand applications with Quality of Service. Meanwhile, modern FPGAs have significantly enhanced their capabilities with advanced manufacturing processes and increased logic elements, able to handle more complex and diverse computational demands [9]–[11]. These improvements facilitate fine-grained FPGA sharing [12]–[15] in multi-user and multi-task environments.

Traditional FPGA virtualization and sharing techniques [7], [16], [17] allocate the entire FPGA to a single application and implement the time-multiplexing by performing a full fabric reconfiguration. This approach introduces significant context switch overhead and often leads to FPGA under-utilization, as applications may not require the full FPGA resources. Instead, *Dynamic Partial Reconfiguration* (DPR) [18] enables an FPGA to split the programmable fabric into multiple *slots*, which can be reconfigured independently to host arbitrary application logic at runtime. It provides the capability to perform *Partial*

Reconfiguration (PR) on a slot while other slots continue running. Currently, DPR has emerged as the most efficient and flexible technique for spatio-temporal FPGA sharing and virtualization [13]–[15] in the data center.

However, current DPR-based spatio-temporal FPGA sharing still contains numerous overlooked challenges, leading to significant application performance degradation. First, in the current FPGA Processing System (PS), the PR module, *PCAP* (Processor Configuration Access Port), is limited to serial bitstream loading. This serial access causes frequent mutual blocking when multiple applications require PR concurrently. As the number of applications sharing an FPGA increases, the PR contention intensifies, resulting in highly prolonged and unpredictable application response times. Meanwhile, the mutual PR blocking further severely disrupts applications' original pipelines, leading application tasks with dependencies in various slots to wait for each other to complete, preventing further execution. Recent systems like Nimblock [15] and DML [14] overlook these problems, resulting in application performance that falls short of the expected pipeline outcomes. Moreover, these works [14], [15] used single-core task scheduling, in which PR operations further block the launching of application tasks, leading to severe task blocking problems.

Second, the uniform size of FPGA slots in previous work [14], [15] imposes strict constraints on task partitioning. For example, when optimizing application tasks using Xilinx's High-Level Synthesis (HLS) tools, resource consumption typically exhibits stepwise increases rather than linear growth, which can easily cause resource over-subscription and under-utilization within slots. While prior work [12] attempted to address the problem by dynamically adjusting slot sizes based on application resource demand, the solution needs regenerating bitstreams at runtime, incurring large runtime overhead.

To address these challenges, we propose **VersaSlot** (Versatile Slots), an efficient spatio-temporal FPGA sharing system, to achieve more effective FPGA multiplexing for applications. To mitigate PR contention among application tasks, VersaSlot introduces a novel *Big.Little* slot architecture. The architecture allows the system to bundle and load multiple tasks simultaneously to a Big slot for internal execution at runtime, thereby eliminating further PR contention with tasks from Little slots. Within the Big slot, tasks can still maintain their parallel or serial pipelines, as they do in the Little slots,

but without internal PR interference. Meanwhile, reduced PR contention also allows Little slots to load task bitstreams promptly, mitigating blocking delays and pipeline disruptions.

However, similar to ARM’s big.LITTLE cores [19], the heterogeneous *Big.Little* slot design introduces increased complexity for resource allocation and scheduling. As for resource allocation, we propose an adaptive algorithm that combines primary allocation and redistribution along with binding and rebinding. The algorithm enables applications to flexibly switch between using Big and Little slots at runtime while also maximizing the utilization of available slot resources. The complementary between Big and Little slots can help mitigate under-utilization and over-subscription problems. In the context of scheduling, the VersaSlot dynamically bundles 3-in-1 tasks at runtime and schedules the corresponding tasks to available slots. The 3-in-1 task bundling can enhance the flexibility in task partitioning of applications. Importantly, VersaSlot decouples the PR from scheduling logic and distributes them across two CPU cores for asynchronous execution, thus effectively eliminating the task execution blocking problem.

For specific applications with few tasks and large batch sizes, PR occurs infrequently. By dividing Big slots into multiple Little slots, more applications can efficiently share resources, making an FPGA with only Little slots (*Only.Little*) also necessary. However, switching an FPGA between *Only.Little* and *Big.Little* configurations requires restarting the system and interrupting all task executions, which introduces a large runtime overhead. To address this, we propose a seamless cross-board switching and live migration mechanism based on system PR contention levels within a cluster, enabling the FPGA slot configuration switching with low overhead.

In a nutshell, the contributions are summarized as follows:

- We propose VersaSlot, an efficient spatio-temporal FPGA sharing system that introduces the novel *Big.Little* slot architecture. VersaSlot addresses the significant PR contention problem inherent in FPGA fine-grained sharing.
- We design an efficient slot allocation and dual-core scheduling algorithm for the *Big.Little* slot architecture to dynamically and efficiently utilize both kinds of slots and prevent the task execution blocking problem.
- We propose a seamless cross-board switching and live migration mechanism, enabling applications to transition among FPGAs with different slot configurations in a cluster for more efficient execution with low overhead.
- We evaluate the VersaSlot system on a real FPGA cluster, against four existing scheduling algorithms. VersaSlot achieves an average response time up to 13.66x lower than the traditional FPGA multiplexing, and up to 2.19x lower than the state-of-the-art system, and enhances the LUT and FF utilization by 35% and 29% on average.

II. BACKGROUND

Dynamic Partial Reconfiguration (DPR), also known as Dynamic Function eXchange (DFX) from AMD/Xilinx [20], is an advanced FPGA configuration technique that enables dynamic reconfiguration of specific regions of the FPGA

without interrupting the overall system operation. Instead, in traditional FPGA configuration flows, any functional modification requires reloading the entire bitstream file, leading to system downtime and a full restart, which causes large runtime overhead. The DFX process relies on the Processor Configuration Access Port (PCAP) [21] to load the partial bitstream from memory to the corresponding section of the FPGA. As it is essential to confirm the successful loading of the partial bitstream, DFX inherently prevents the PCAP from loading a new partial bitstream until the current one is fully loaded and blocks the corresponding CPU [21]. Meanwhile, to avoid unpredictable impacts on the remaining FPGA circuitry during the loading phase, a dedicated IP, DFX Decouple, is employed to decouple the reconfigurable region from the rest of the circuit. Recently, the DFX has been extensively used in FPGA virtualization and sharing in the cloud [13]–[15], [22], but limited to serial partial bitstream loading.

III. SYSTEM DESIGN

A. VersaSlot System Framework

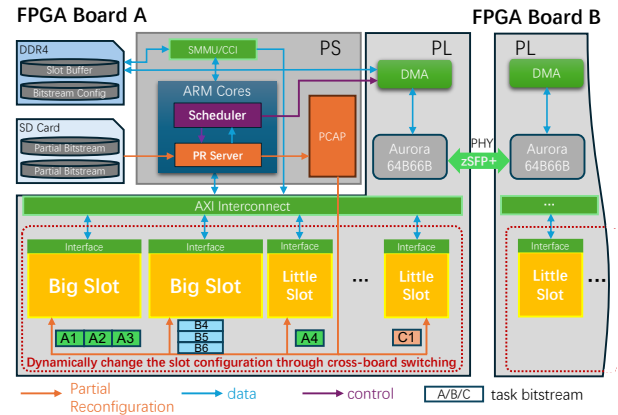


Fig. 1: VersaSlot system with PS and PL in FPGA cluster.

The VersaSlot system, aligned with the typical structure of FPGA sharing design [14], [15], consists of two main components: the embedded hypervisor within the Processing System (PS) and the FPGA fabric layout in the Programmable Logic (PL), as illustrated in Figure 1. The VersaSlot hypervisor in the PS is implemented on ARM cores via bare-metal programming to minimize the system control overhead. Its primary responsibilities include scheduling and resource allocation, managing the loading of partial bitstreams and application data, and coordinating cross-board switching. For FPGA boards without a dedicated CPU, the hypervisor can run on the host CPU and control the FPGA via the PCIe interface.

The VersaSlot hypervisor in the PS comprises two key modules: the scheduler and the PR server, both of which run as bare-metal applications on the ARM cores. At runtime, the scheduler allocates computational slot resources to each incoming application, updates the status of reconfigurable slots, and determines the next application to execute. The PR server, which handles partial reconfiguration, waits for PR requests from the scheduler. It loads pre-generated bitstreams from the SD card into memory and sends commands to the FPGA’s

PCAP module to dynamically switch partial bitstreams for applications. However, the PCAP can only perform PR for one bitstream at a time and suspends the associated CPU during the process. Additionally, the PR server communicates status updates to the scheduler through the On-Chip Memory (OCM). The ARM cores utilize the System Memory Management Unit (SMMU) for address mapping and the AXI Interconnect to transfer application data to the FPGA slots for execution.

In the PL, based on the DPR technique, FPGA resources are divided into a *static region* and partial reconfigurable Big and Little slots. The static region configures interfaces for the reconfigurable slots, enabling slots to communicate with the PS and memory via the AXI bus. The region can only be programmed once at system startup, while the reconfigurable parts can be dynamically reconfigured and switched to map different application logic at runtime. Before execution, application bitstreams are prepared offline in advance. Specifically, applications are partitioned into smaller tasks suitable for Little slots by synthesis resources via automated scripts. The *task* represents a portion of the application and the basic execution unit for a slot. The VersaSlot system supports the dynamic *batch* processing for applications. The fine granularity and batched structure of tasks facilitate an application to be organized in a pipeline with dependencies across slots during execution, which can increase parallelism, thus reducing execution time and enhancing resource utilization. A *slot* refers to a reconfigurable region. Traditionally, the slots are designed to be uniform. In the VersaSlot system, we introduce a novel *Big.Little* slot architecture, where three consecutive small tasks can be bundled as a big task for execution in a Big slot. In our system, an FPGA consists of either 2 Big slots and 4 Little slots (*Big.Little*) or 8 Little slots (*Only.Little*), but can be extended to any Big/Little configuration. The resource capacity of each Big slot is twice that of a Little slot. To ensure slot compatibility, the automated script generates partial bitstreams for each task adaptive to each slot. These generated bitstreams are stored on the SD card. Once partial reconfiguration is complete, the scheduler allocates buffers and launches the task.

The cross-board switching module is another important component in the PL, designed for board switching between different FPGA slot configurations. It utilizes the GT transceivers (zSFP+) to connect other FPGA boards. We introduce the Aurora IP core, an efficient communication protocol to transfer tasks, application information, and data directly via DMA to another FPGA unit when switching is activated.

B. Big.Little Slots

The *Big.Little* architecture is a heterogeneous slot design for spatio-temporal FPGA sharing, coupling resource-intensive Big slots and standard-resource Little slots. Although Little slots can increase resource granularity for more applications to share an FPGA, they also bring severe mutual PR contention and task execution blocking problems due to inherent DPR limitation in serial bitstream loading and CPU blocking. For example, as shown in Figure 2, two applications (App-1 and App-2), each with 3 tasks (T1, T2, T3) and batch sizes of 3

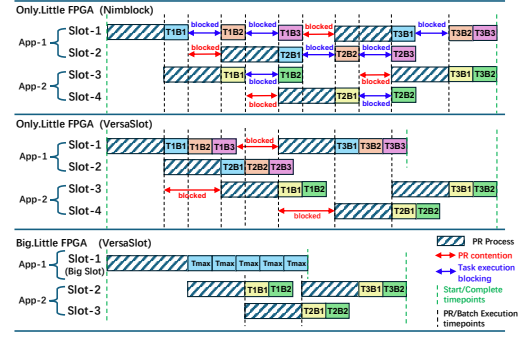


Fig. 2: Versaslot with *Big.Little* and *Only.Little* dual-core scheduling alleviates the PR contention and task execution blocking problems, thus reducing application response time.

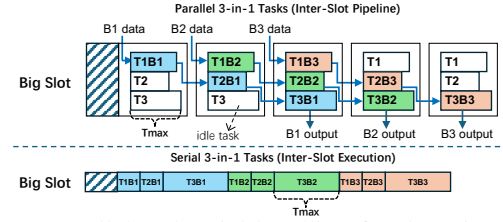


Fig. 3: Parallel and serial bundling for the 3-in-1 task.

(B1, B2, B3) and 2 (B1, B2), are processed on an FPGA with 4 Little slots, in which each application is allocated with 2 slots that forms the execution pipeline. In the Nimblock [15], after the App-1 completes the PR of its first task, the App-2 starts the PR of its first task, during which it blocks the bitstream loading of App-1's second task (T2) although App-1's first batch item of the first task (T1B1) has finished the execution. Subsequently, the PR of App-1's second task (T2) also blocks the PR of App-2's second task. Moreover, since Nimblock handles all operations and scheduling by a single CPU core, the PR process of App-2's T1 also blocks the task execution of App-1's T1B2, and the PR of App-1's T2 blocks the task execution of App-2's T1B2. As the two applications continue running, mutual PR contention and task execution blocking alternate, disrupting each application's original pipeline and significantly increasing response times. The figure shows only 4 slots with 2 applications, but as the number of shared slots and applications increases, this effect becomes more severe.

Within our uniform Little slots (*Only.Little*) FPGA design, VersaSlot decouples PR logic from the scheduler to a dedicated PR server on a separate CPU core, which effectively resolves task execution blocking. As in Figure 2, while other applications perform PR operations, the scheduler can still promptly trigger batch execution of tasks without waiting for specific PR completion points. Additionally, we pre-load tasks of the same application into the slots to further enhance pipeline efficiency. The *Only.Little* design is suited for specific applications with few tasks and large batch sizes, as PR occurs infrequently. But PR contention still persists in most workloads.

Therefore, we propose the Big slots, which can dynamically bundle and load 3 tasks (3-in-1 task) of an application simultaneously for internal execution at runtime. As shown in Figure 2, with VersaSlot's *Big.Little* slots, once App-1 and its tasks

are bundled and scheduled to execute on the Big slot, App-1 only needs to trigger tasks' batch execution without frequent PR loading, thus avoiding outside PR blocking from Little slots. Meanwhile, App-2's tasks can complete its pipeline smoothly without the PR blocking from App-1. Thus, the PR contention of applications in both Big and Little slots is alleviated. Furthermore, as in Figure 3, the tasks within Big slots can be organized in a pipeline as well. Since each batch's parallel execution time on the Big slot equals the longest of the three tasks ($T_{\max}$), serial execution is preferable when $T_{\max} \cdot (N_{\text{batch}} + 2) > \sum (T_1 + T_2 + T_3) \cdot N_{\text{batch}}$. With this criterion, the VersaSlot system can select the optimal 3-in-1 task bitstream for execution at runtime. We set the bundling size to be 3 based on the Big slot's resource capacity to accommodate tasks and its fewer idle task cycles in pipelines than a larger size.

C. Slot Allocation and On-board Scheduling

Similar to the ARM big.LITTLE cores [19], the heterogeneous *Big.Little* slot architecture introduces greater complexity in resource allocation and scheduling. We propose an efficient slot allocation and scheduling algorithm for the architecture. When an application A_i enters the candidate list, it is first added to the waiting list C_{wait} for slot allocation. Once the application acquires slot resources R_{A_i} (the maximum number of slots to use), all of its tasks are pushed into the ready list Q_T , awaiting scheduling for execution in Big or Little slots.

1) *Slot Allocation*: As detailed in Algorithm 1, slot allocation is based on available slots and system status, featuring the processes of primary allocation and redistribution along with binding and rebinding. In the primary allocation, the system first allocates slots for applications in the waiting list C_{wait} . It prioritizes allocating Big slots to applications that can bundle tasks, as Big slots can significantly mitigate PR contention. After the Big slot allocation, Little slots are assigned to applications based on the optimal slot count $O_{A_i}^L$. The optimal slot allocation, derived through integer linear programming (ILP) as in [14], [15], represents the most efficient slot configuration for pipeline execution within either Little or Big slots, whose value is usually lower than the task count. The primary allocation ensures applications execute promptly along their task pipelines. When additional Little slots are left, VersaSlot performs the redistribution based on applications already assigned with Little slots, prioritizing those at the front of the runnable queue to obtain the maximum needed slots. This redistribution effectively avoids slot idling and maximizes overall slot utilization. Meanwhile, to prevent situations where Big slots are idle while Little slots are overburdened, we propose a rebinding method for load balancing. For applications assigned to Little slots but not yet executing, we unbind them for new slot allocation, returning them to a waiting state. This ensures that Big slots can receive new applications for execution. But applications bound to the big slots can only complete all their tasks in the Big slots to avoid Big slot blocking caused by task dependencies.

2) *On-board Scheduling*: Algorithm 2 outlines the overall scheduling process, which primarily includes 3-in-1 task on-line bundling, task execution launching, and scheduling tasks

Algorithm 1: Slot Allocation

Input: $C_{\text{wait}} = \{A_i\}$: apps A_i waiting for slot allocation;
 $S_{\text{Big}} = \{A_i\}$: the list of apps allocated with Big slots;
 $S_{\text{Little}} = \{A_i\}$: the list of apps allocated with Little slots;
 B_{avail} : idle Big slots; L_{avail} : idle Little slots; (number);
 Q_T : the list of ready tasks waiting for scheduling;
 $N_{T_{A_i}}$: the number of unfinished ready tasks of A_i ;
 $O_{A_i} = (O_{A_i}^B, O_{A_i}^L)$: optimal Big/Little slots for A_i 's pipeline
Output: $R_{A_i} = (R_{A_i}^B, R_{A_i}^L)$: Big/Little slots allocated to A_i

```

1  $B_{\text{avail}} = B_{\text{total}} - \sum N_{T_{A_i}}, A_i \in S_{\text{Big}} \wedge T_{A_i} \in Q_T$ ;
   // No slot available, skip the allocation.
2 if  $B_{\text{avail}} \leq 0$  and  $L_{\text{avail}} \leq 0$  then
3   return;
   // Unbind apps with Little slots for rebinding.
4 foreach  $A_i \in S_{\text{Little}}$  and  $B_{\text{avail}} > 0$  do
5   if not  $\text{isAppStarted}(A_i)$  then
6      $S_{\text{Little}} = S_{\text{Little}} \setminus \{A_i\}$ ;  $C_{\text{wait}} = C_{\text{wait}} \cup \{A_i\}$ ;
   // Primary allocation.
7  $L_{\text{left}} = L_{\text{total}} - \sum \min(R_{A_i}^L, N_{T_{A_i}}), A_i \in S_{\text{Little}} \wedge T_{A_i} \in Q_T$ ;
   foreach  $A_i \in C_{\text{wait}}$  do
   // Binding, prioritize big slot allocation.
8   if  $B_{\text{avail}} > 0$  and  $\text{canBundle}(A_i)$  then
9      $R_{A_i} = (O_{A_i}^B, 0)$ ;  $S_{\text{Big}} = S_{\text{Big}} \cup \{A_i\}$ ;
10     $B_{\text{avail}} = B_{\text{avail}} - 1$ ; continue;
   // Binding, allocation with Little slots.
11  if  $L_{\text{avail}} > 0$  and  $L_{\text{left}} > 0$  then
12     $R_{A_i} = (0, O_{A_i}^L)$ ;  $S_{\text{Little}} = S_{\text{Little}} \cup \{A_i\}$ ;
13     $L_{\text{left}} = L_{\text{left}} - R_{A_i}^L$ ;
   // Redistribution for left Little slots.
14 if  $L_{\text{left}} > 0$  then
15   foreach  $A_i \in S_{\text{Little}}$  and  $L_{\text{left}} > 0$  do
16      $\delta = N_{T_{A_i}} - O_{A_i}^L, T_{A_i} \in Q_T$ ;
17      $R_{A_i} = (0, \min(L_{\text{left}} + R_{A_i}^L, \delta + R_{A_i}^L))$ ;
18      $L_{\text{left}} = L_{\text{left}} - \delta$ ;
19 return  $\{R_{A_i}\}$ , updated  $S_{\text{Big}}, S_{\text{Little}}, C_{\text{wait}}$ ;
```

to their designated slots for PR. For newly added tasks in the ready list Q_T assigned with Big slots, the scheduler bundles them into serial or parallel 3-in-1 tasks at runtime and replaces them with the bundled tasks in the ready list for execution. Meanwhile, the scheduler updates task execution statuses, loads task data, and launches batch executions. In previous work [14], [15], this step was often blocked by subsequent PR processes, leading to the task execution blocking problem. However, VersaSlot employs dual-core scheduling, which decouples the PR process and scheduling logic onto two separate CPU cores. When the scheduler assigns a new task to its corresponding available slot for PR, it sends an asynchronous request to the PR server without waiting for PR completion. Meanwhile, the scheduler limits application slot usage to its allocation, ensuring efficient spatio-temporal FPGA sharing. In VersaSlot's *Only.Little* system, the scheduler follows the preemption mechanism as in [15], to prevent long-running tasks from monopolizing the FPGA. But the preemption brings more PR operations. The *Big.Little* applies preemption only in Little slots since an application cannot occupy both Big and Little slots simultaneously. Meanwhile, the inherent design of the slot redistribution in *Big.Little* also avoids monopolization.

Algorithm 2: Scheduling Algorithm

Input: $S_{Big} = \{A_i\}$, $S_{Little} = \{A_i\}$, $\{R_{A_i}\}$;
 B_{avail} : idle Big slots; L_{avail} : idle Little slots; (number);
 Q_T : the list of ready tasks waiting for scheduling;
 $U_{A_i} = (U_{A_i}^B, U_{A_i}^L)$: number of Big/Little slots used by A_i ;
 // Add new tasks to the ready task list.

```

1 foreach  $A_i \in S_{Big} \cup S_{Little}$  do
2   if  $T_{A_i} \notin Q_T$  then
3      $Q_T.push\_back(T_{A_i})$ ;

  // Bundle 3-in-1 task for Big slots online.
4 foreach  $A_i \in S_{Big}$  and  $\{T_{A_i}\} \subset Q_T$  do
5   if not  $isBundled(\{T_{A_i}\})$  then
6      $T_{A_i}^{bundled} = 3in1Bundle(\{T_{A_i}\})$ ;
7      $Q_T$  replaces  $\{T_{A_i}\}$  with  $\{T_{A_i}^{bundled}\}$ ;

  // Launch batch execution for running tasks.
8 foreach  $T_{A_i} \in Q_T$  do
9   if  $isRunning(T_{A_i})$  and  $waitBatchExec(T_{A_i})$  then
10     $launch(T_{A_i})$ ;
11  else if  $isFinished(T_{A_i})$  then
12     $Q_T = Q_T \setminus T_{A_i}$ ;  $isFinished(A_i) \Rightarrow S = S \setminus A_i$ ;

  // Schedule tasks to corresponding slots.
13 foreach  $T_{A_i} \in Q_T$  do
14   if  $A_i \in S_{Big}$  and  $U_{A_i}^B < R_{A_i}^B$  and  $B_{avail} > 0$  then
15     PR( $T_{A_i}$ ) to an idle Big slot (Async in PR server);
16      $T_{A_i}.running = True$ ;  $B_{avail} = B_{avail} - 1$ ;
17   if  $A_i \in S_{Little}$  and  $U_{A_i}^L < R_{A_i}^L$  and  $L_{avail} > 0$  then
18     PR( $T_{A_i}$ ) to an idle Little slot (Async in PR server);
19      $T_{A_i}.running = True$ ;  $L_{avail} = L_{avail} - 1$ ;

```

D. Cross-Board Switching and Live Migration

1) *Live Migration*: As mentioned in section III-B, *Only.Little* and *Big.Little* configurations have their own advantages for different application workloads, requiring an FPGA to switch between them to maximize performance. However, configurations related to slot size and interfaces are located in the *static region*, which can only be programmed once during system start-up. Cross-board switching enables seamless transitions between configurations without rebooting, achieved through live workload migration. When switching is triggered, the original FPGA stops executing new tasks. Applications and tasks in the ready list, along with their buffers, are transferred via DMA to a new pre-configured FPGA. Once transferred, the new FPGA resumes task execution and processes upcoming new workloads. Meanwhile, ongoing tasks on the original FPGA continue to completion to avoid bitstream reloading overhead, and the FPGA is freed afterward to prevent excess resource usage. Therefore, in a cluster, a single available FPGA can enable cross-board switching for the entire system.

2) *Performance Degradation Metric D_{switch}* : To determine the optimal timing for the switching and evaluate system performance degradation from PR contention, we propose an evaluation metric, D_{switch} , considering all influencing factors.

$$D_{switch} = \frac{N_{blocked_tasks}}{N_{PR}} \times \frac{N_{apps}}{N_{batch}}, \quad (0 < D_{switch} < 1) \quad (1)$$

where $N_{PR} = \sum_{A_i} N_{T_{A_i}}$, $A_i \in R_c \cup R_s$; $N_{batch} = \sum_{A_i} B_{A_i}$, $A_i \in C$. The metric D_{switch} combines the ratio $\frac{N_{blocked_tasks}}{N_{PR}}$ to evaluate current PR contention degree among

tasks, and the ratio $\frac{N_{apps}}{N_{batch}}$ to assess potential conflicts from candidate applications. It is normalized from 0 to 1, where higher values indicate more severe contention and less predictable application response time. The metric is recalculated after every n updates to the application candidates queue with total completed (R_c) / running (R_s) / ready (R_r) applications. The $N_{blocked_task}$ represents the number of tasks blocked by PR contention during this period. The N_{PR} denotes the number of PR tasks $\{T_{A_i}\}$ by all completed and running applications A_i . The $N_{blocked_tasks}$ is positively correlated with N_{PR} , which indicates a higher number of PR typically results in more blocked tasks and an increasing D_{switch} . Another two influential factors, batch size B_{A_i} and the number of applications N_{apps} , are significantly factored into the future contention evaluation. The N_{batch} is the total batch size of all applications A_i in the current application candidate queue C . If each application is allocated only one slot with batch size to be one, $N_{batch} = N_{apps}$, this indicates the worst-case scenario for PR conflicts and corresponds to the maximum value of D_{switch} .

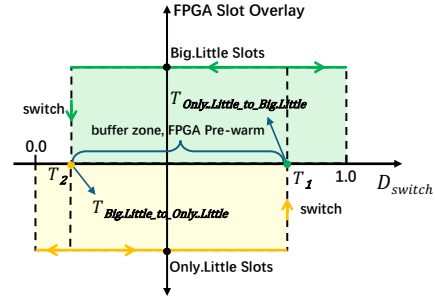


Fig. 4: Cross-board switching loop with the buffer zone.

Inspired by Schmitt Trigger [23], we propose a switch loop for seamless cross-board switching across multiple FPGAs. As shown in Figure 4, as the D_{switch} increases and reaches the threshold T_1 , the system triggers the switching from the *Only.Little* slots to the *Big.Little* slots, thereby alleviating the PR contention. Similarly, when the D_{switch} decreases to the threshold T_2 , it triggers the switching from the *Big.Little* FPGA back to *Only.Little* FPGA, allowing more future applications to share an FPGA. The two user-configurable thresholds adjust switching sensitivity, with a buffer zone to prevent frequent switching and ensure system stability. Meanwhile, when the D_{switch} metric enters the buffer zone, the system anticipates the direction of change, pre-configures the potential FPGA configuration, and loads task bitstreams into SD storage in a new FPGA. The pre-warming design further ensures seamless live migration with low overhead.

IV. EXPERIMENT AND EVALUATION

The VersaSlot system was evaluated on an FPGA cluster with two latest Xilinx UltraScale+ FPGAs (ZCU216). We used the same benchmark as previous work [15] and partitioned applications into multiple tasks based on the optimal fit between slot resources and task resource usage after synthesis. The applications include 3D Rendering (3DR, 3 tasks), LeNet (6 tasks), Image Compression (IC, 6 tasks), AlexNet (AN, 6 tasks), and Optical Flow (OF, 9 tasks). The partitioning and task bitstream generation were performed automatically

by a TCL script in Vivado 2024.1. We compared the VersaSlot system with four existing scheduling algorithms/systems, including the traditional exclusive temporal multiplexing (Baseline) [7], [16], First-come-first-served spatio-temporal sharing (FCFS), round-robin (RR) from [22], and the state-of-the-art spatio-temporal sharing system Nimblock [15]. To simulate different congestion conditions, we randomly generated application workloads (10 sequences, 20 apps/sequence) with random batch size (5-30) and arrival intervals, including Loose (5000ms), Standard (1500ms-2000ms), Stress (150ms-200ms), and Real-time (50ms) for the evaluation.

A. Response Time Reduction and Tail Latency

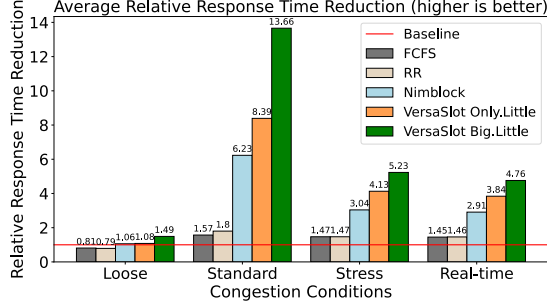


Fig. 5: Relative response time reduction under different congestion conditions, normalized to the baseline.

As shown in Figure 5, we analyze the relative response time reduction under different congestion conditions. The VersaSlot *Big.Little* design outperforms all other methods across all congestion conditions. Under standard workloads, it outperforms the baseline up to 13.66x. The FCFS and Round-robin perform slightly better than the baseline. Compared to the state-of-the-art system, Nimblock, the *Big.Little* design reduces average response time up to 2.17x in standard workloads, 1.72x under stress, and 1.63x in real-time conditions, showcasing the efficiency of *Big.Little* architecture and dual-core scheduling in fine-grained spatio-temporal FPGA sharing. Additionally, compared to the VersaSlot *Only.Little* configuration, *Big.Little* achieves average response time improvements by 63%, 27%, and 24%, respectively, verifying its effectiveness in alleviating PR contention. Meanwhile, as shown in Figure 6, we further analyze the tail latency of applications, and the *Big.Little* design consistently outperforms Nimblock in P95 and P99 metrics (95th and 99th percentiles) across all congestion conditions. Under stress workloads, *Big.Little* improves these metrics by 83% and 46%, respectively, and by 56% and 48% under real-time workloads. While *Big.Little* shows a slight increase in P99 tail latency compared to the baseline, it maintains or even improves P95 performance. Given its substantial advantage in response time, *Big.Little* obviously demonstrates superior application performance than the baseline.

B. Resource Utilization

Figure 7 shows the resource utilization improvement of 3-in-1 tasks in Big slots compared to running all tasks only in Little slots. All applications show significant LUT and FF utilization increases, with average improvements of 35% and 29%. Specifically, the right figure displays the LUT usage for

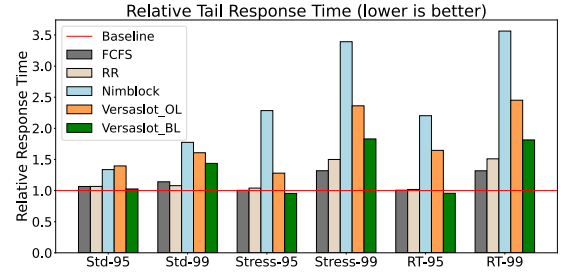


Fig. 6: Tail response time normalized to the baseline.

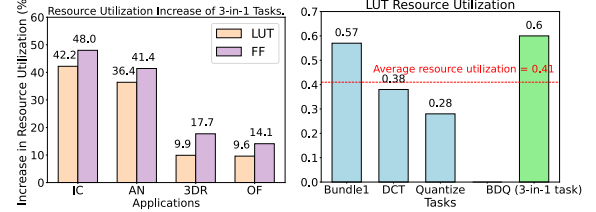


Fig. 7: Resource utilization improvement by 3-in-1 tasks.

the first three tasks of application IC and their bundled 3-in-1 task. Since task partitioning is normally based on synthesis resource usage, Bundle1's LUT utilization decreases from 0.98 in synthesis to 0.57 in implementation. But, with bundling, the 3-in-1 task improves average utilization from 0.41 to 0.6.

C. Cross-board Switching

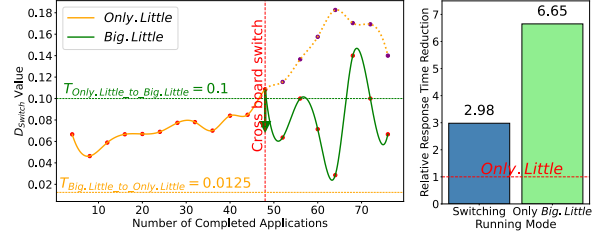


Fig. 8: D_{switch} and relative response time with switching.

To validate the effectiveness of cross-board switching, we tested three long workloads, each with 80 applications and standard arrival intervals. Figure 8 (left) shows the D_{switch} variation for one workload every 4 application updates, where the D_{switch} value triggers a switch from *Only.Little* to *Big.Little* at the threshold, enabling tasks to execute on the new *Big.Little* FPGA. Compared to execution solely on *Only.Little*, cross-board switching reduces the average response time up to nearly 3x with the average switching overhead of 1.13ms.

V. CONCLUSION

This paper presents VersaSlot, an efficient spatio-temporal FPGA sharing system with the novel *Big.Little* slot architecture to address the PR contention problem. We propose a slot allocation and dual-core scheduling algorithm and a seamless cross-board switching and live migration mechanism for the architecture to maximize FPGA multiplexing in the cluster. VersaSlot achieves up to 2.19x average response time improvement over the state-of-the-art system and enhances the LUT and FF utilization by 35% and 29% on average.

ACKNOWLEDGMENT

We sincerely thank Leibniz Supercomputing Centre (LRZ) and Munich Quantum Valley (MQV) for the FPGA support.

REFERENCES

- [1] E. Chung, J. Fowers, K. Ovtcharov, M. Papamichael, A. Caulfield, T. Massengill, M. Liu, D. Lo, S. Alkalay, M. Haselman, M. Abeydeera, L. Adams *et al.*, “Serving dnns in real time at datacenter scale with project brainwave,” *IEEE Micro*, vol. 38, no. 2, pp. 8–20, 2018.
- [2] T. Wang, L. Gong, C. Wang, Y. Yang, Y. Gao, X. Zhou, and H. Chen, “Via: A novel vision-transformer accelerator based on fpga,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 11, pp. 4088–4099, 2022.
- [3] C. Wu, T. Geng, A. Guo, S. Bandara, P. Haghi, C. Liu, A. Li, and M. Herbordt, “Fasda: An fpga-aided, scalable and distributed accelerator for range-limited molecular dynamics,” in *SC23: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–14.
- [4] S. Zeng, Z. Zhu, J. Liu, H. Zhang, G. Dai, Z. Zhou, S. Li, X. Ning, Y. Xie, H. Yang, and Y. Wang, “Df-gas: a distributed fpga-as-a-service architecture towards billion-scale graph-based approximate nearest neighbor search,” in *2023 56th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023, pp. 283–296.
- [5] J. Zhao, Y. Zhang, J. Cheng, Y. Wu, C. Ye, H. Yu, Z. Huang, H. Jin, X. Liao, L. Gu, and H. Liu, “Sagrap: A similarity-aware hardware accelerator for temporal graph processing,” in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–6.
- [6] Alibaba Cloud, “What is FPGA as a Service? - FPGA as a Service,” <https://www.alibabacloud.com/help/en/fpga-as-a-service/product-overview/what-is-fpga-as-a-service>, [Accessed 27-10-2024].
- [7] Amazon AWS EC2, “Amazon EC2 F1 Instances,” <https://aws.amazon.com/ec2/instance-types/f1/>, [Accessed 27-10-2024].
- [8] Microsoft Azure, “Microsoft Azure Stack Edge Pro FPGA overview,” <https://learn.microsoft.com/en-us/azure/databox-online/azure-stack-edge-overview>, [Accessed 27-10-2024].
- [9] C. Bobda, J. M. Mbongue, P. Chow, M. Ewais, N. Tarafdar, J. C. Vega, K. Eguro, D. Koch, S. Handagala, M. Leiser, M. Herbordt, H. Shahzad, P. Hofste, B. Ringlein, J. Szefer, A. Sanaullah, and R. Tessier, “The future of fpga acceleration in datacenters and the cloud,” *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, vol. 15, no. 3, Feb. 2022.
- [10] A. Damiani, G. Fisaletti, M. Bacis, R. Brondolin, and M. D. Santambrogio, “Blastfunction: A full-stack framework bringing fpga hardware acceleration to cloud-native applications,” *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, vol. 15, no. 2, pp. 1–27, 2022.
- [11] B. Ringlein, F. Abel, D. Diamantopoulos, B. Weiss, C. Hagleitner, M. Reichenbach, and D. Fey, “A case for function-as-a-service with disaggregated fpgas,” in *2021 IEEE 14th International Conference on Cloud Computing (CLOUD)*, 2021, pp. 333–344.
- [12] A. Khawaja, J. Landgraf, R. Prakash, M. Wei, E. Schkufza, and C. J. Rossbach, “Sharing, protection, and compatibility for reconfigurable fabric with AmorphOS,” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. Carlsbad, CA: USENIX Association, Oct. 2018, pp. 107–127.
- [13] Y. Zha and J. Li, “Virtualizing fpgas in the cloud,” in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS ’20. New York, NY, USA: Association for Computing Machinery, 2020, p. 845–858.
- [14] A. Dhar, E. Richter, M. Yu, W. Zuo, X. Wang, N. S. Kim, and D. Chen, “DML: Dynamic Partial Reconfiguration With Scalable Task Scheduling for Multi-Applications on FPGAs,” *IEEE Transactions on Computers*, vol. 71, no. 10, pp. 2577–2591, Oct. 2022.
- [15] M. Mandava, P. Reckamp, and D. Chen, “Nimblock: Scheduling for fine-grained fpga sharing through virtualization,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA ’23. New York, NY, USA: Association for Computing Machinery, 2023.
- [16] A. Putnam, A. M. Caulfield, E. S. Chung, D. Chiou, K. Constantinides, J. Demme, H. Esmaeilzadeh, J. Fowers, G. P. Gopal, J. Gray, M. Haselman, S. Hauck, S. Heil, A. Hormati, J.-Y. Kim, S. Lanka, J. Larus, E. Peterson, S. Pope, A. Smith, J. Thong, P. Y. Xiao, and D. Burger, “A reconfigurable fabric for accelerating large-scale datacenter services,” in *2014 ACM/IEEE 41st International Symposium on Computer Architecture (ISCA)*, 2014, pp. 13–24.
- [17] N. Tarafdar, T. Lin, E. Fukuda, H. Bannazadeh, A. Leon-Garcia, and P. Chow, “Enabling flexible network fpga clusters in a heterogeneous cloud data center,” in *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA ’17. New York, NY, USA: Association for Computing Machinery, 2017, p. 237–246.
- [18] Xilinx, “Dynamic function exchange,” <https://docs.amd.com/v/u/2020.1-English/ug909-vivado-partial-reconfiguration>, [Accessed 08-11-2024].
- [19] ARM big.LITTLE, “Arm Cortex-a Series, Global Task Scheduling,” <https://developer.arm.com/documentation/den0013/d/big-LITTLE/Software-execution-models-in-big-LITTLE/Global-Task-Scheduling>, [Accessed 08-10-2024].
- [20] AMD Xilinx, “AMD Technical Information Portal - dynamic function exchange,” <https://docs.amd.com/r/en-US/ug909-vivado-partial-reconfiguration/Introduction>, [Accessed 25-10-2024].
- [21] AMD, “AMD Technical Information Portal - pcap,” <https://docs.amd.com/r/en-US/ug1085-zynq-ultrascale-trm/Initialize-PCAP-Interface>, [Accessed 25-10-2024].
- [22] D. Korolija, T. Roscoe, and G. Alonso, “Do OS abstractions make sense on FPGAs?” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 991–1010. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/roscoe>
- [23] “Schmitt-Trigger – Wikipedia — de.wikipedia.org,” <https://de.wikipedia.org/wiki/Schmitt-Trigger>, [Accessed 30-10-2024].